

Esquemas de Detección y Corrección

Integrantes:

Mathew Cordero – 22982

Javier Chen – 22153

Descripción de la Práctica:

Este laboratorio se enfocó en la implementación de un sistema de transmisión y recepción de mensajes utilizando una arquitectura por capas. El sistema simula un canal de comunicación no confiable, en el cual se pueden introducir errores mediante una capa de “ruido”. Para manejar estos errores, se integraron algoritmos de detección (como CRC-32) y corrección (como Hamming).

El objetivo general fue experimentar cómo se comportan estos esquemas al transmitir información expuesta a interferencias, y cómo los distintos algoritmos pueden detectar o corregir errores, dependiendo de su capacidad.

Metodología Utilizada:

El desarrollo se organizó en cinco capas funcionales:

- **Capa de Aplicación:** Solicita el mensaje a enviar y el algoritmo de integridad; en el receptor, muestra el mensaje decodificado o un error si no es recuperable.
- **Capa de Presentación:** Convierte el mensaje a binario ASCII y lo decodifica en la recepción.
- **Capa de Enlace:** Agrega o verifica la información de integridad según el algoritmo elegido (CRC o Hamming). Aplica corrección si es posible.
- **Capa de Ruido:** Simula errores bit a bit con una probabilidad configurable, afectando incluso bits de paridad.
- **Capa de Transmisión:** Encargada de enviar y recibir tramas mediante sockets.

Se desarrollaron emisores y receptores separados en distintos lenguajes, de acuerdo con los lenguajes usados para los algoritmos ya implementados.

Como ejecutar los programas:

Como ejecutar conector.java

Archivos requeridos:

- Connector.java
- Coder.java
- CRC32Emisor.java
- AnalizadorTramas.java
- GeneralizedHammingDecoder.java
- Decoder.java
- .env

Estructura del .env:

- HOST=127.0.0.1
- HAMMING_PORT=8000
- CRC_PORT=9000

Copilar el programa:

```
javac *.java
```

Como ejecutar el programa

Modo Receptor:

```
java Connector haming
```

Modo Emisor:

```
java Connector crc
```

Como ejecutar conector.py

Archivos requeridos:

- conector.py
- emisorhamming.py
- receptorcrc32.py
- .env

Como ejecutar el programa:

Modo Emisor:

```
python conector.py emisor
```

Modo Receptor:

python conector.py receptor

Resultados:

```
Trama Recibida:
[1, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1]"
Datos decodificados: [0, 1, 1, 0, 0, 0, 0, 1]
Tipo de error: SINGLE_ERROR_CORRECTED
Posición del error: 6
Mensaje: Error simple corregido en la posición 6
Trama corregida: [1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1]
Mensaje Decodificado: a
PS E:\Laboratorio2-Redes>

===
Ingrese el mensaje a enviar
Trama codificada: 110111010
ing) ===
Ingrese el mensaje a enviar: a
Mensaje enviado correctamente usando Hamming.
Mensaje original: a
Trama codificada: 110111010001
Trama con ruido: 110110010001
PS E:\Laboratorio2-Redes>
```

Imagen 1: Receptor en Java y Emisor en Python de Hamming funcinando

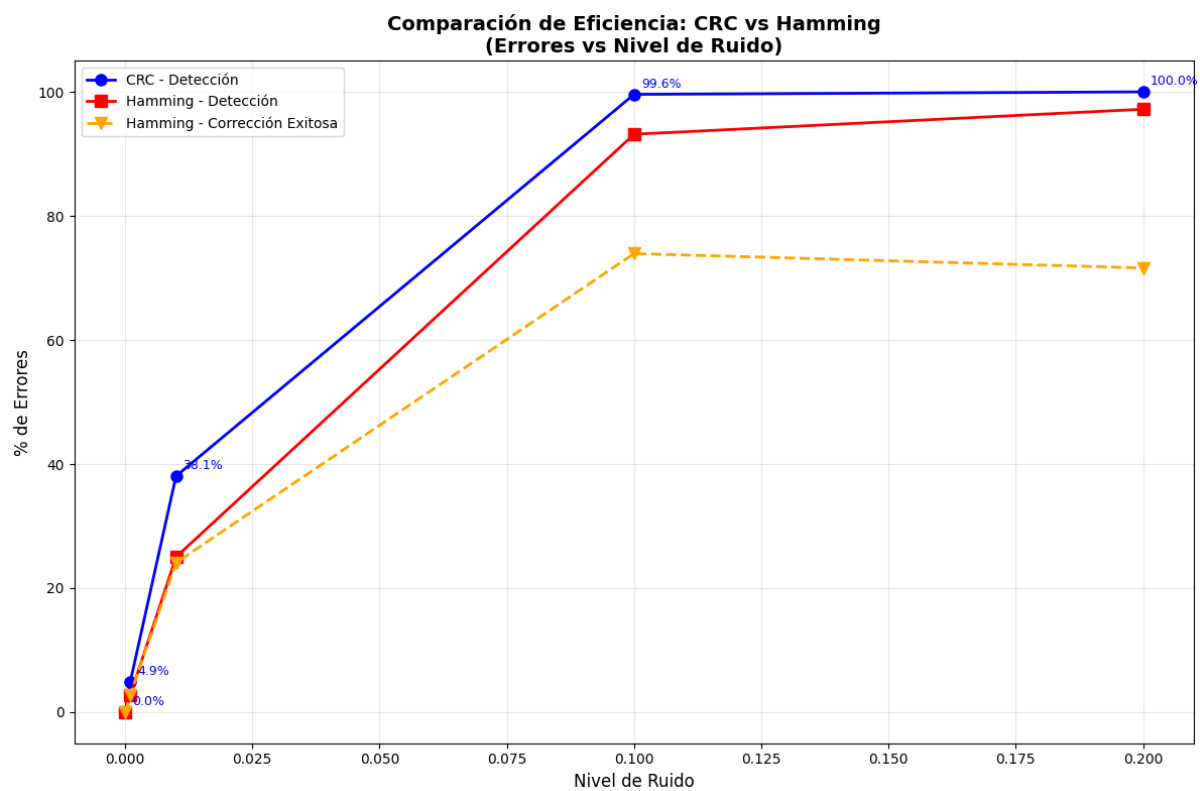
```
PS E:\Laboratorio2-Redes> java Connector crc
=== MODO EMISOR (CRC) ===
Conectando a 127.0.0.1:9000
Conexión establecida
Escriba su mensaje: a
Mensaje original: a
Trama original: 0110000110101000011001001101101100100000
Trama con ruido: 0110000110101000011001001101101100100000
Mensaje enviado exitosamente

PS E:\Laboratorio2-Redes> python conector.py receptor
=== MODO RECEPTOR (Sólo CRC) ===
Escuchando en 127.0.0.1:9000...
Conexión entrante desde ('127.0.0.1', 54220)
Trama recibida: 0110000110101000011001001101101100100000
Mensaje recibido correctamente (CRC): a
PS E:\Laboratorio2-Redes>
```

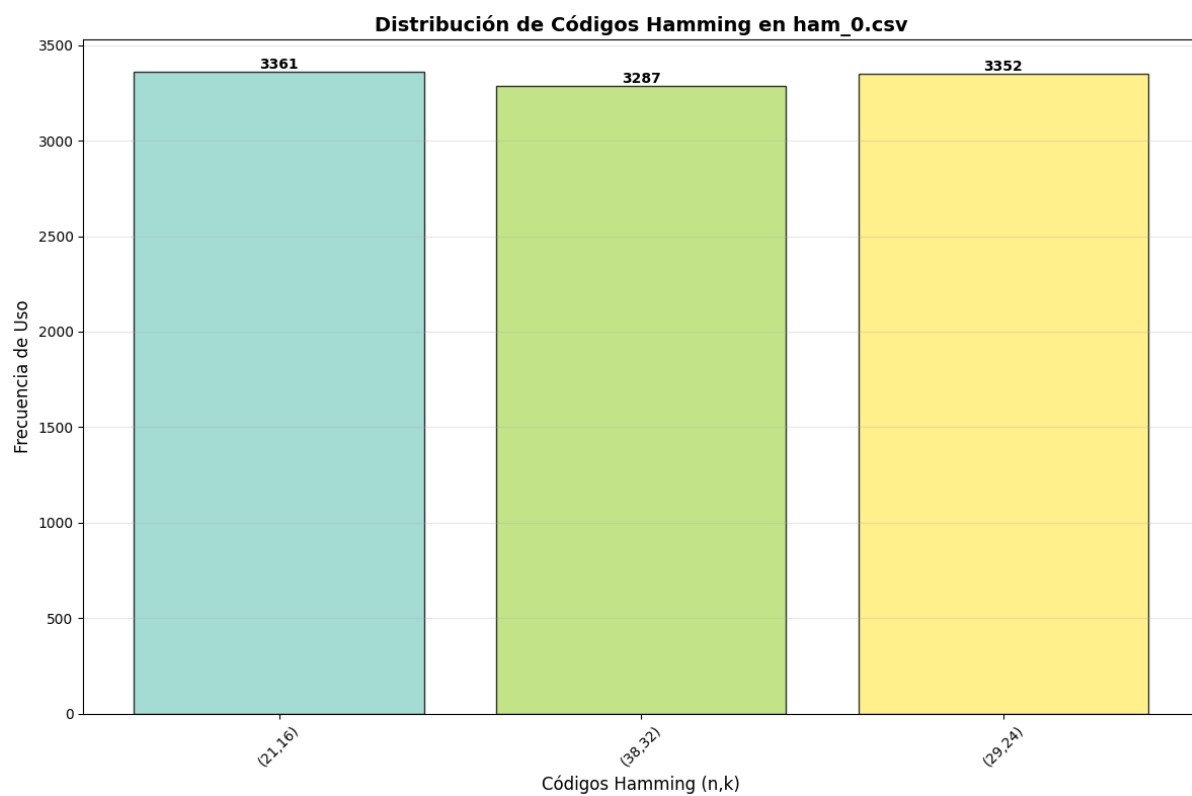
Imagen 2: Receptor en Python y Emisor en Java de CRC funcinando

Nombre del archivo	Total, registros	Porcentaje de detección	Porcentaje de corrección	Porcentaje sin errores
ham_0.csv	10000	0%	0.00%	100.00%
ham_0.001.csv	10000	2.73%	2.73%	97.27%
ham_0.01.csv	10000	25.02%	24.12%	74.98%
ham_0.1.csv	10000	93.16%	74.46%	6.84%
ham_0.2.csv	10000	97.20%	72.41%	2.80%

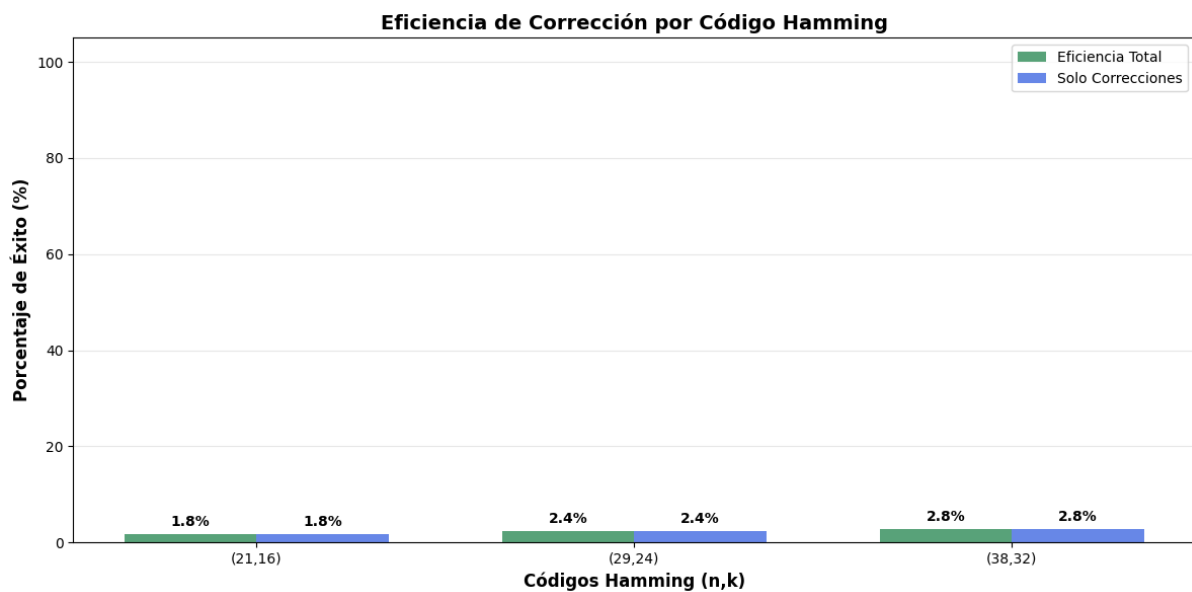
Tabla 1: Porcentaje de detección error Hamming



Grafica 1 : Corrección y Detección de Errores para 10000 tokens



Grafica 2 : Hamming mas usados por los tokens



Grafica 3 : Deteccion y correccion de errores por Hamming usado

Discusión de sus resultados y pruebas:

Como se puede observar los 2 algoritmos lograron detectar errores en la grafica 1, pero de entre todos el que mejor le fue fue al CRC. La grafica muestra cuantos errores fueron detectados o corregidos exitosamente y que si eran errores. Como se ve el CRC logro detectar mas errores que el Hamming y la correccion no hamming se puede ver incluso que se estanco y no mejoro mucho con respecto a su valor anterior . Esto indica que el hamming no es tan bueno detectando errores, pero corrigiendolos lo hace bien , aunque si existe demasiado ruido, por la naturaleza del mismo, no logra corregir bien.

De entre todos se observo que el que tuvo mayor tasa de aceptacion de errores fue el CRC , esto porque el hamming realmente solo deja detectar 2 , corregir 1 y el resto de errores no los detecta. Debido a esto lo hace un algoritmo menos flexible , pero eso si bastante bueno para usarse en ciertos contextos. Pues realmente la diferencia de usar uno u otro dependera la situacion y el contexto. Es bueno hacer uso de un mecanismo de correccion solo si nuestro sistema no se permite fallar y tambien si el mensaje no es tan largo. Como se observa en la figura 3 los hamming con n, m muy altos provocaron menores aciertos.

En cambio si nuestra comunicacion es confiable ya sea teniendo ciertos mecanismos de aislamiento ante el ruido como fibra optica, etc. O los mensajes tienden a ser muy largos, al igual que se puede retransmitir de manera facil la informacion para ignorar la

ultima trama es mejor utilizar un mecanismo de solo deteccion. Pues nos garantiza que, si o sí, el mensaje llegará íntegro, ya que si se detecta un error, simplemente se descarta la trama y se solicita una retransmisión. Esto no siempre es posible con un mecanismo de corrección, el cual puede intentar recuperar datos corruptos, pero no siempre con éxito.

Además, los algoritmos de corrección suelen ser computacionalmente más costosos, ya que requieren realizar cálculos adicionales para identificar y reconstruir los datos originales, lo cual implica mayor uso de recursos y tiempo de procesamiento. Esta diferencia se vuelve crítica en sistemas donde la rapidez y el bajo consumo de recursos son prioritarios, como en comunicaciones en tiempo real o dispositivos con hardware limitado.

Comentario grupal sobre el tema y sus hallazgos:

La práctica fue una experiencia interesante ya que nos permitió aplicar conceptos teóricos de redes en un entorno práctico y funcional. Implementar un sistema de transmisión completo por capas nos ayudó a entender mejor cómo fluye la información desde la aplicación hasta el medio físico, y cómo se puede controlar la integridad del mensaje en cada etapa. Uno de los mayores aprendizajes fue coordinar el funcionamiento entre distintas capas, especialmente al integrar la codificación, simulación de ruido, y la comunicación por sockets.

Conclusiones

- El mejor algoritmo con mayor rendimiento es el de deteccion por CRC32
- Es mas flexible tener CRC ya que no tiene las limitaciones de hamming a la cantidad de bits por paridad
- Es mejor usar un algoritmo de correccion solo si nuestro medio es poco fiable y no podemos retransmitir en todo caso siempre es mejor uno de solo detección.

Referencias

GitHub: <https://github.com/JavierC22153/Laboratorio2-Redes/tree/lab2.2>

